

Day 06: ユーザー登録画面を作ろう

今日のゴール

新規ユーザーがアカウントを作成できる登録画面を作ります。名前、メールアドレス、パスワード、パスワード確認の入力欄を用意し、バリデーションを実装します。

【スクリーンショット: 完成したユーザー登録画面】

なぜこれを作るのか？

ログインするには、まずアカウントが必要です。ユーザー登録画面を作ることによって、新しいユーザーがアプリを使い始められるようになります。また、パスワードの確認入力やバリデーションを実装することで、セキュリティも向上します。

例え話: お店の会員登録と同じです。名前や連絡先を登録することで、次回から簡単にログインできるようになります。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	登録ページを作成	10分
Step 2	フォームを実装	15分
Step 3	パスワード確認を追加	10分
Step 4	APIと連携	15分

合計時間: 約50分

Step 1: 登録ページを作成（10分）

ゴール: `/register` ページを作成して、登録フォームのベースを作ります。

実装:

```
// filepath: src/app/register/page.tsx
'use client';

import { Box, Typography } from '@mui/material';

export default function RegisterPage() {
  return (
    <Box
      sx={{
        maxWidth: 400,
        mx: 'auto',
        mt: 8,
        p: 3,
      }}
    >
      <Typography variant="h4" sx={{ mb: 3 }}>
        新規登録
      </Typography>
    </Box>
  );
}
```

確認ポイント: `/register` にアクセスして「新規登録」が表示される

学んだこと: Day 5と同じパターンで、新しいページを作成できた

Step 2: フォームを実装 (15分)

ゴール: 名前、メール、パスワードの入力欄を追加します。

実装:

```
// filepath: src/app/register/page.tsx
'use client';

import { useState } from 'react';
import { Box, TextField, Button, Typography } from '@mui/

export default function RegisterPage() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState(

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log({ name, email, password });
  };

  return (
    <Box sx={{ maxWidth: 400, mx: 'auto', mt: 8, p: 3 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>
        新規登録
      </Typography>
      <form onSubmit={handleSubmit}>
        <TextField
          fullWidth
          sx={{ mb: 2 }}
          label="名前"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        <TextField
          fullWidth
          sx={{ mb: 2 }}
          label="メールアドレス"
          type="email"
          value={email}
```

```

        onChange={(e) => setEmail(e.target.value)}
      />
      <TextField
        fullWidth
        sx={{ mb: 2 }}
        label="パスワード"
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <TextField
        fullWidth
        sx={{ mb: 2 }}
        label="パスワード (確認)"
        type="password"
        value={confirmPassword}
        onChange={(e) => setConfirmPassword(e.target.value)}
      />
      <Button fullWidth type="submit" variant="contained">
        登録
      </Button>
    </form>
  </Box>
);
}

```

確認ポイント: 4つの入力欄が表示される

学んだこと: 複数のuseStateを使って、複数の入力欄を管理できた

Step 3: パスワード確認を追加 (10分)

ゴール: パスワードと確認用パスワードが一致するかチェックします。

実装:

```
// filepath: src/app/register/page.tsx (バリデーション部分のみ)
const [errors, setErrors] = useState({
  name: '',
  email: '',
  password: '',
  confirmPassword: '',
});

const validate = () => {
  const newErrors = {
    name: '',
    email: '',
    password: '',
    confirmPassword: '',
  };

  if (!name) {
    newErrors.name = '名前を入力してください';
  }

  if (!email || !/\S+@\S+\.\S+/.test(email)) {
    newErrors.email = 'メールアドレスの形式が正しくありません';
  }

  if (!password || password.length < 8) {
    newErrors.password = 'パスワードは8文字以上で入力してください';
  }

  if (password !== confirmPassword) {
    newErrors.confirmPassword = 'パスワードが一致しません';
  }

  setErrors(newErrors);
  return Object.values(newErrors).every(error => !error);
};
```

バリデーションルール:

項目	ルール
名前	必須
メール	必須 + メール形式
パスワード	必須 + 8文字以上
パスワード確認	パスワードと一致

確認ポイント: パスワードが一致しないとエラーが表示される

学んだこと: 複数の条件を組み合わせたバリデーションを実装できた

Step 4: APIと連携（15分）

ゴール: tRPCを使って、ユーザー登録APIを呼び出します。

実装:

```
// filepath: src/app/register/page.tsx
import { api } from '@trpc/react';
import { useRouter } from 'next/navigation';

export default function RegisterPage() {
  const router = useRouter();
  const registerMutation = api.auth.register.useMutation({
    onSuccess: () => {
      router.push('/login');
    },
  });

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (validate()) {
      registerMutation.mutate({ name, email, password });
    }
  };

  return (
    // ... フォームは同じ
  );
}
```

確認ポイント: 登録後に/loginにリダイレクトされる

学んだこと: tRPCのmutationを使って、APIを呼び出せるようになった

今日のまとめ

- ☐ ユーザー登録ページを作成できた
- ☐ 複数の入力欄を管理できた
- ☐ パスワード確認のバリデーションを実装できた
- ☐ tRPCでAPIを呼び出せた

⚠ つまづきポイント

エラー/問題	原因	解決方法
パスワードが一致しない	比較ロジックが間違っている	<code>password !== confirmPassword</code> で比較
API呼び出しが失敗する	tRPCの設定が間違っている	<code>api.auth.register</code> のパスを確認

次回予告

Day 7では、実際のログイン処理を実装します。NextAuthを使って、セッション管理を行います。